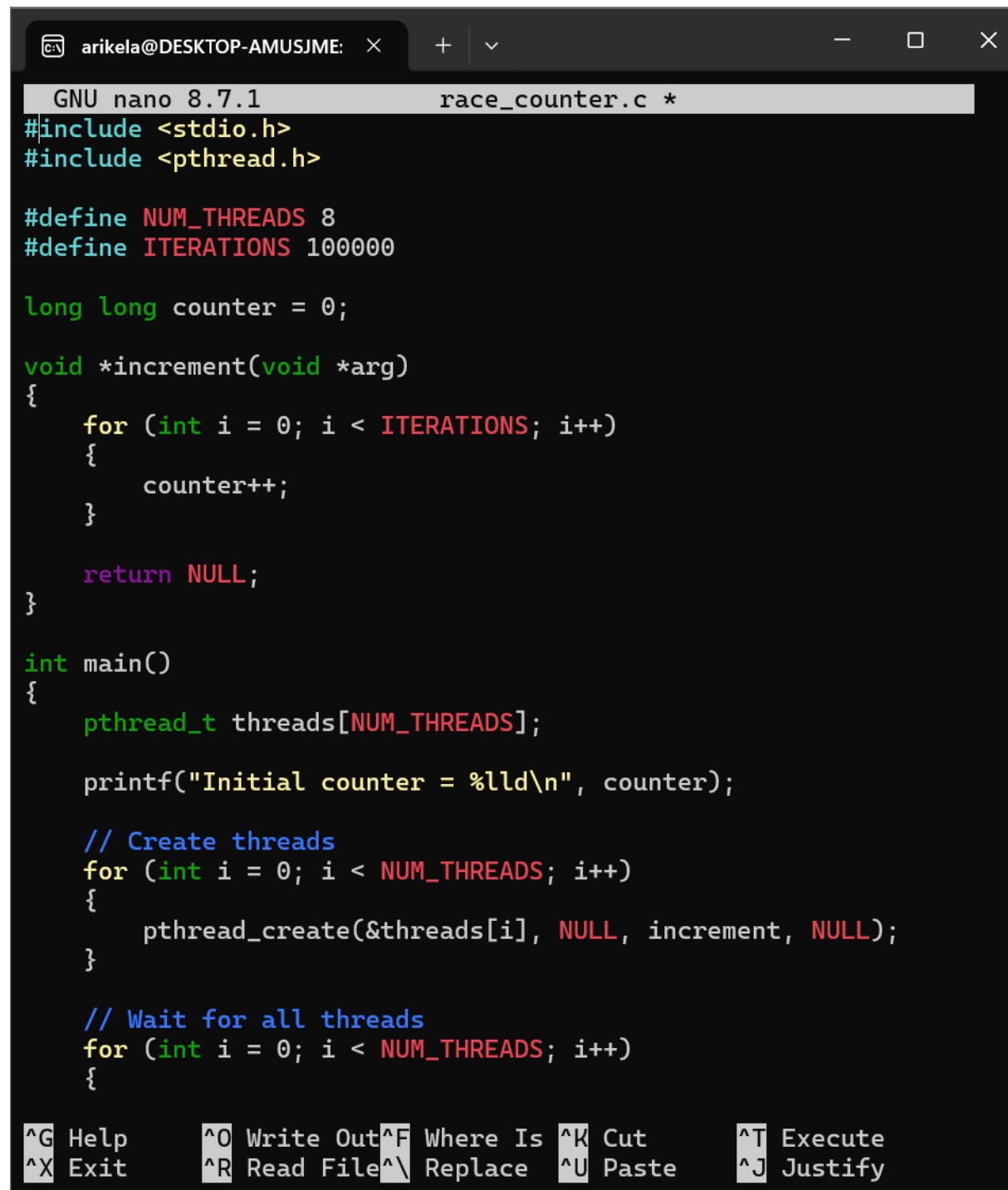


Practical Week-11

Task-1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.



```
GNU nano 8.7.1 race_counter.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 8
#define ITERATIONS 100000

long long counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Initial counter = %lld\n", counter);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter = %lld\n", counter);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

// Wait for all threads
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Final counter = %lld\n", counter);
printf("Expected counter = %lld\n",
       (long long)NUM_THREADS * ITERATIONS);

return 0;
}
|

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```

arikel@DESKTOP-AMUSJME:~/week5prac$ nano race_counter.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc race_counter.c -o race_
counter -pthread
arikel@DESKTOP-AMUSJME:~/week5prac$ ./race_counter
Initial counter = 0
Final counter = 800000
Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$ ./race_counter
Initial counter = 0
Final counter = 616208
Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$ ./race_counter
Initial counter = 0
Final counter = 560012
Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$

```

Task-2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

arikela@DESKTOP-AMUSJME: ×

+ v

— □ ×

GNU nano 8.7.1

race_counter.c *

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 8
#define ITERATIONS 100000

long long counter = 0;

pthread_mutex_t mutex;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&mutex);

        counter++;

        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&mutex, NULL);

    printf("Initial counter = %lld\n", counter);

    // Create threads
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```

// Create threads
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_create(&threads[i], NULL, increment, NULL);
}

// Wait for all threads
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Final counter = %lld\n", counter);
printf("Expected counter = %lld\n",
        (long long)NUM_THREADS * ITERATIONS);

pthread_mutex_destroy(&mutex);

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$ nano race_counter.c
arikel@DESKTOP-AMUSJME:~/week5prac$ nano race_counter.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc race_counter.c -o mutex
_counter -pthread
arikel@DESKTOP-AMUSJME:~/week5prac$ ./mutex_counter
Initial counter = 0
Final counter = 800000
Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$ ./mutex_counter
Initial counter = 0
Final counter = 800000
Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$ ./mutex_counter
Initial counter = 0
Final counter = 800000
Expected counter = 800000
arikel@DESKTOP-AMUSJME:~/week5prac$ |

```